

PROGRAMACIÓN III · PROF. PULJIZ

BrasilPagos

Documentación técnica — Trabajo Práctico Integrador

INTEGRANTES

Agustín Valussi
Rios Fabrizio

UTN — Facultad Regional Resistencia
2026

La documentación complementaria del TP (consignas, flujos, wireframes y diseño Figma) se encuentra en la carpeta `ESTRUCTURA` de la entrega, junto a este PDF. Este documento cubre específicamente la estructuración técnica y funcional de la app BrasilPagos: arquitectura, modelo de datos, instalación, ejecución y API.

BrasilPagos

Documentación técnica — App mobile (React Native + Expo) para comparar cotizaciones ARS→BRL entre billeteras virtuales argentinas para pagos vía PIX en Brasil.

1. Arquitectura general



Flujo de autenticación

La app usa **dos sistemas de auth en paralelo**: Firebase Auth controla el acceso a la app (exige email verificado), y un JWT propio controla el acceso a los endpoints protegidos de la API.

- **Login email/contraseña:** Firebase login → chequea `emailVerified` → `POST /api/auth/login` → JWT → guardado en AsyncStorage.
- **Login con Google:** Firebase Sign-In con ID token → `POST /api/auth/firebase-login` → JWT propio.
- **App reabierta / sin token:** `onAuthStateChanged` detecta sesión Firebase activa; si no hay JWT guardado, se pide uno nuevo con `getIdToken()`.

2. Stack técnico

Frontend (app mobile)

Tecnología	Versión
React Native	0.81.5
Expo SDK	54
React	19.1.0
React Navigation	6.x
Firebase Auth	email/contraseña + Google

Backend (API REST)

Tecnología	Versión
Node.js	22.x
Express	4.x
MySQL	8.x
mysql2	pool de conexiones
JWT / bcryptjs	auth + hash de contraseñas

3. Modelo de datos

```
usuarios
├─ alertas (1:N)
├─ historial_consultas (1:N)
├─ favoritos (1:N)

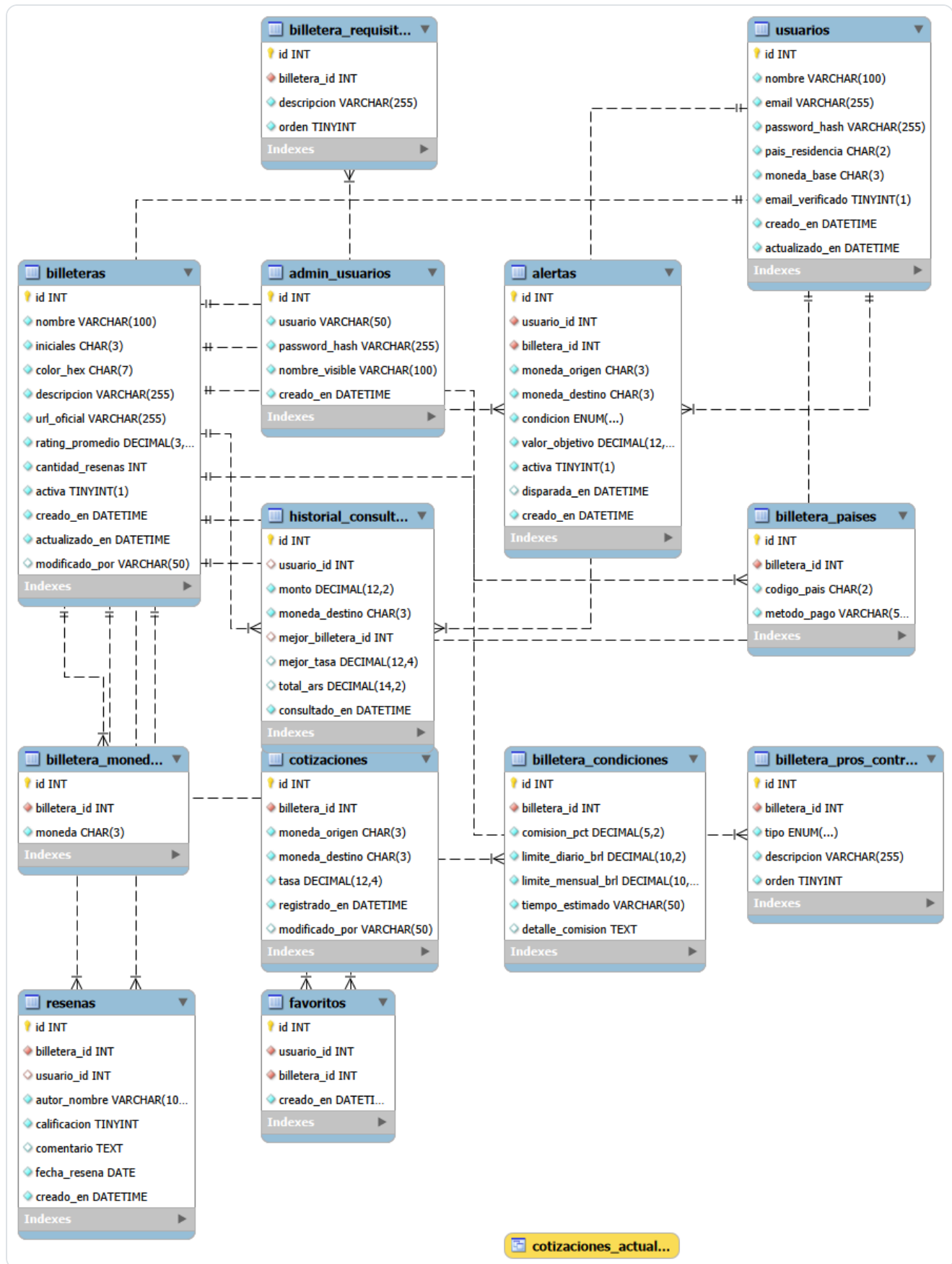
billeteras ← MAESTRO principal
├─ billetera_paises (1:N)      países destino soportados
├─ billetera_monedas (1:N)    monedas aceptadas
├─ billetera_condiciones (1:1) límites y comisiones
├─ billetera_pros_contras (1:N) pros y contras
├─ billetera_requisitos (1:N) requisitos de apertura
├─ cotizaciones (1:N)         historial de tasas ARS/BRL
├─ reseñas (1:N)              opiniones de usuarios

admin_usuarios ← login del panel admin (fabri / agus), independiente de "usuarios"
```

VIEW `cotizaciones_actuales` : devuelve la última tasa registrada por billetera, ordenada de mejor a peor. La API la consulta directamente para armar el ranking.

Auditoría: `billeteras` y `cotizaciones` tienen columna `modificado_por` (+ `actualizado_en` en `billeteras`), completada automáticamente con el admin logueado en cada mutación desde el panel.

Diagrama Entidad-Relación (MER)



4. Instalación paso a paso

4.1 Requisitos previos

Herramienta	Versión
Node.js	22.x
MySQL + MySQL Workbench	8.x / reciente
Expo Go (celular)	última versión
Android SDK / adb	para el método USB

4.2 Clonar e instalar dependencias

```
git clone <url-del-repo>
cd ComparadorBilleteras
npm install

cd server
npm install
```

4.3 Configurar la base de datos

En MySQL Workbench, ejecutar en este orden:

```
SOURCE database/brasilpagos_schema.sql;  -- tablas, FK, índices, VIEW
SOURCE database/brasilpagos_datos.sql;    -- 13 billeteras + 2 admin_usuarios
```

Si la base ya existe (no se recrea desde cero), correr además la migración aditiva para sumar el login de admin y las columnas de auditoría:

```
SOURCE database/migracion_admin_auth.sql;
```

4.4 Variables de entorno del servidor

Crear `server/.env` :

```
DB_HOST=localhost
DB_USER=root
DB_PASSWORD=tu_password
DB_NAME=brasilpagos
JWT_SECRET=una_clave_secreta_larga
PORT=3000
```

4.5 (Opcional) Firebase Admin SDK

Para habilitar la gestión de usuarios Firebase desde el panel admin, copiar el archivo de credenciales en `server/serviceAccountKey.json` (se descarga desde Firebase Console → Configuración → Cuentas de servicio → Generar clave privada). Sin este archivo, `/api/auth/firebase-login` usa la API pública de Google como fallback automático.

5. Cómo iniciar la app — paso a paso

5.1 Levantar el servidor (API REST)

```
cd server
node index.js
# API en http://localhost:3000
# Panel admin en http://localhost:3000/admin
```

5.2 Método recomendado: celular físico por USB + adb reverse

Funciona en cualquier red, sin depender del WiFi.

1. Conectar el celular por USB con **Depuración USB** activada (Ajustes → Acerca del teléfono → tocar 7 veces "Número de compilación" → Opciones de desarrollador → Depuración USB: ON).
2. Verificar que el dispositivo aparece:

```
adb devices
```

3. Hacer el reverse del puerto (repetir si se desconecta el cable):

```
adb -s <ID_DISPOSITIVO> reverse tcp:3000 tcp:3000
```

4. Levantar Metro:

```
npx expo start -c
```

5. Escanear el QR con Expo Go desde el celular.

La URL en `src/services/api.js` debe ser `http://localhost:3000` (valor por defecto, no requiere cambios).

5.3 Alternativa: misma red WiFi

```
npx expo start -c
# o, si hay problemas de red:
npx expo start --tunnel --clear
```

5.4 Build de APK (opcional)

```
cd C:\dev\CB\android
.\gradlew.bat app:assembleDebug
adb install -r C:\dev\CB\android\app\build\outputs\apk\debug\app-debug.apk
```

Gradle debe mantenerse en versión **8.3** (versiones 8.8+ rompen expo-modules-core). El build se compila desde el junction `C:\dev\CB\android` para evitar errores de CMake por rutas largas en Windows.

6. API REST — resumen de endpoints

Base URL: `http://localhost:3000`

Públicos (sin token)

Método	Ruta	Descripción
GET	<code>/api/cotizaciones?monto=500</code>	Ranking de billeteras para el monto dado
GET	<code>/api/cotizaciones/historial?billetera_id=1</code>	Historial de tasas de una billetera (hasta 30 registros)
GET	<code>/api/billeteras?nombre=&pais=</code>	Listado de billeteras activas, con filtro opcional por nombre y país (resuelto en SQL)
GET	<code>/api/billeteras/:id</code>	Detalle completo: condiciones, pros/contras, requisitos, países, reseñas
POST	<code>/api/auth/register</code>	Crear cuenta <code>{ nombre, email, password }</code>
POST	<code>/api/auth/login</code>	Login → <code>{ token, usuario }</code>
POST	<code>/api/auth/firebase-login</code>	Canjea ID token de Firebase por JWT propio
POST	<code>/api/admin/login</code>	Login del panel admin → JWT con <code>rol: 'admin'</code> (8h)

Protegidos (`Authorization: Bearer <token>`)

Método	Ruta	Descripción
GET	<code>/api/auth/me</code>	Datos del usuario autenticado
GET/POST/PATCH/DELETE	<code>/api/alertas</code>	CRUD de alertas de cotización
GET	<code>/api/historial?page=&limit=</code>	Historial de consultas paginado real (LIMIT/OFFSET en DB)
POST	<code>/api/historial</code>	Guardar una consulta
GET/POST/DELETE	<code>/api/favoritos</code>	Gestión de billeteras favoritas
POST	<code>/api/resenas</code>	Crear reseña; recalcula rating en transacción

Admin (Authorization: Bearer <token admin>)

Método	Ruta	Descripción
POST	/api/cotizaciones	Cargar nuevas tasas (graba <code>modificado_por</code>)
GET	/api/admin/stats	Estadísticas del dashboard
GET	/api/admin/billeteras	Todas las billeteras, sin filtrar por activa
PATCH	/api/admin/billeteras/:id/toggle	Mostrar/ocultar billetera
PUT	/api/admin/billeteras/:id/rating	Actualizar rating y cantidad de reseñas
GET/POST/PUT/DELETE	/api/admin/usuarios	CRUD de usuarios (MySQL + Firebase sync)
GET	/api/admin/firebase-usuarios	Unión Firebase + MySQL

7. Seguridad

- **Contraseñas:** hasheadas con `bcryptjs` (salt 10). Nunca se guarda texto plano.
- **JWT de usuario:** firmado con `JWT_SECRET`, expira en 7 días. Verificado por `middleware/auth.js` en cada ruta protegida.
- **Firebase Auth:** exige email verificado para poder usar la app.
- **Panel admin:** login real contra la tabla `admin_usuarios` (solo dos cuentas fijas, `fabri` / `agus`, sin endpoint de alta). Devuelve JWT con `rol: 'admin'`, validado por `middleware/adminAuth.js`. Reemplaza la vieja clave compartida `X-Admin-Key`.
- **Auditoría:** cada mutación admin graba automáticamente `modificado_por` con el usuario logueado.

8. Panel admin

Accesible en `http://localhost:3000/admin`. Requiere login contra la tabla `admin_usuarios`. Incluye:

- **Dashboard** — estadísticas generales (usuarios, billeteras activas, consultas del día)
- **Cotizaciones** — carga masiva de tasas para todas las billeteras
- **Billeteras** — toggle visible/oculta, edición de rating, columna "Modificado por"
- **Usuarios** — CRUD completo con sincronización Firebase (requiere `serviceAccountKey.json`)

8.1 Credenciales de acceso admin

Solo existen dos cuentas fijas, sin endpoint de alta (a propósito — cada integrante del grupo opera el panel con su propia identidad):

Usuario	Contraseña	Rol
fabri	admin	admin
agus	admin	admin

El login se hace en `POST /api/admin/login` con `{ usuario, password }`, que devuelve un JWT con `{ adminUsuario, rol: 'admin' }` válido por 8 horas. Ese token se guarda en `localStorage` del panel y se

envía como `Authorization: Bearer <token>` en cada request admin (verificado por `middleware/adminAuth.js`).

Contraseñas hasheadas con bcrypt en la tabla `admin_usuarios`. Si la base ya existía antes de esta feature, hay que correr `database/migracion_admin_auth.sql` para crear la tabla y cargar estas dos cuentas.

9. Pantallas principales (23 implementadas)

Módulo	Pantallas
Onboarding	SplashScreen, OnboardingScreen, LoadingSplashScreen
Auth	LoginScreen, RegisterScreen, EmailVerificationScreen, ForgotPasswordScreen
Principal	HomeScreen, ResultsScreen, EmptyResultsScreen, LoadingResultsScreen
Billeteras	WalletsScreen, WalletProfileScreen (perfil + gráfico de tasa + reseñas), WalletCompareScreen
Alertas	AlertsScreen, CreateAlertScreen, PushNotificationScreen (polling cada 60s)
Perfil	ProfileScreen, EditProfileScreen, HistoryScreen (paginado), FavoritesScreen, SettingsScreen
Global	ErrorScreen

10. Comandos útiles — resumen rápido

```
# Servidor
cd server && node index.js

# App mobile - Expo Go
npx expo start -c
npx expo start --tunnel --clear # desde red distinta al servidor

# adb reverse (celular por USB)
adb devices
adb -s <ID> reverse tcp:3000 tcp:3000

# Build APK (desde el junction)
cd C:\dev\CB\android
.\gradlew.bat app:assembleDebug
```